

EXPERIMENT NO 3:

Demonstrate the working of the decision tree based ID3 algorithm. Use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample.

Aim:

To implement and demonstrate the ID3 (Iterative Dichotomiser 3) Decision Tree Algorithm using a training dataset and classify a new sample.

Procedure:

1. Define the training dataset in the program.
2. Separate the features and the target class.
3. Encode the categorical data into numerical values.
4. Train the Decision Tree model using the Entropy criterion (ID3).
5. Display the generated decision tree.
6. Predict the class label for a new sample.
7. Display the predicted result.

Program:

```
# ID3 Decision Tree Algorithm
```

```
import pandas as pd
```

```
from sklearn.preprocessing import LabelEncoder
```

```
from sklearn.tree import DecisionTreeClassifier, export_text
```

```
# Training Dataset
```

```
data = {
```

```
    'Outlook': ['Sunny', 'Sunny', 'Overcast', 'Rain', 'Rain',
```

```
               'Rain', 'Overcast', 'Sunny', 'Sunny', 'Rain',
```

```
               'Sunny', 'Overcast', 'Overcast', 'Rain'],
```

```
    'Temperature': ['Hot', 'Hot', 'Hot', 'Mild', 'Cool',
```

```
                   'Cool', 'Cool', 'Mild', 'Cool', 'Mild',
```

```
                   'Mild', 'Mild', 'Hot', 'Mild'],
```

```
    'Humidity': ['High', 'High', 'High', 'High', 'Normal',
```

```

        'Normal', 'Normal', 'High', 'Normal', 'Normal',
        'Normal', 'High', 'Normal', 'High'],
    'Wind': ['Weak', 'Strong', 'Weak', 'Weak', 'Weak',
             'Strong', 'Strong', 'Weak', 'Weak', 'Weak',
             'Strong', 'Strong', 'Weak', 'Strong'],
    'Play': ['No', 'No', 'Yes', 'Yes', 'Yes',
             'No', 'Yes', 'No', 'Yes', 'Yes',
             'Yes', 'Yes', 'Yes', 'No']
}

df = pd.DataFrame(data)

# Encode categorical data

le = LabelEncoder()

X = df.drop('Play', axis=1)

y = le.fit_transform(df['Play'])

encoders = {}

for col in X.columns:

    encoders[col] = LabelEncoder()

    X[col] = encoders[col].fit_transform(X[col])

# Train ID3 Decision Tree

model = DecisionTreeClassifier(criterion='entropy')

model.fit(X, y)

# Print Decision Tree

print("Decision Tree:\n")

print(export_text(model, feature_names=list(X.columns)))

# New sample for prediction

new_sample = pd.DataFrame({

    'Outlook': ['Sunny'],

    'Temperature': ['Cool'],

    'Humidity': ['High'],

    'Wind': ['Strong']
})

```

```
}}
```

```
for col in new_sample.columns:
```

```
    new_sample[col] = encoders[col].transform(new_sample[col])
```

```
prediction = model.predict(new_sample)
```

```
print("\nPrediction:")
```

```
print("Play =", le.inverse_transform(prediction)[0])
```

Output:

```
*** Decision Tree:
|--- Outlook <= 0.50
|   |--- class: 1
|--- Outlook > 0.50
|   |--- Humidity <= 0.50
|   |   |--- Outlook <= 1.50
|   |   |   |--- Wind <= 0.50
|   |   |   |   |--- class: 0
|   |   |   |   |--- Wind > 0.50
|   |   |   |   |   |--- class: 1
|   |   |   |--- Outlook > 1.50
|   |   |   |   |--- class: 0
|   |   |--- Humidity > 0.50
|   |   |   |--- Wind <= 0.50
|   |   |   |   |--- Temperature <= 1.00
|   |   |   |   |   |--- class: 0
|   |   |   |   |   |--- Temperature > 1.00
|   |   |   |   |   |   |--- class: 1
|   |   |   |--- Wind > 0.50
|   |   |   |   |--- class: 1
|   |--- class: 1
|--- class: 1

Prediction:
Play = No
```

Result:

The ID3 Decision Tree Algorithm was successfully implemented using the given dataset. A decision tree was generated based on Information Gain (Entropy), and the new sample was successfully classified.

Predicted Class: Play = No